PRACTICAL FILE

OF

DATABASE MANAGEMENT SYSTEMS LABARATORY

(UGCA 1925)

Submitted By:

Bikkar Singh

2447072

BCA-4th Sem

Submitted To:

Mr. Navjot Singh

Assistant Professor

Department of Computer Application

PUNJAB INSTITUTE OF MANAGEMENT AND TECHNOLOGY

(AFFILIATED TO IKG PUNJAB TECHNICAL UNIVERSITY, JALANDHAR)

10	Consider the following schema for Order Database: SALESMAN (Salesman_id, Name, City, Commission) CUSTOMER (Customer_id, Cust_Name, City, Grade, Salesman_id) ORDERS (Ord_No, Purchase_Amt, Ord_Date, Customer_id, Salesman_id) Write SQL queries to 1. Count the customers with grades above Amritsar's average. 2. Find the name and numbers of all salesmen who had more than one customer. 3. List all salesmen and indicate those who have and don't have customers in their cities (Use UNION operation.) 4. Create a view that finds the salesman who has the customer with the highest order of a day. 5. Demonstrate the DELETE operation by removing salesman with id 1000. All his orders must also be deleted.	26-33	
11	Write a PL/SQL code to add two numbers and display the result. Read the numbers during run time.	34	
12	Write a PL/SQL code to find sum of first 10 natural numbers using while and for loop.	35-36	
13	Write a program to create a trigger which will convert the name of a student to upper case before inserting or updating the name column of student table.	37-38	
14	Write a PL/SQL block to count the number of rows affected by an update statement using SQL%ROWCOUNT	39-40	
15	Write a PL/SQL block to increase the salary of all doctors by 1000.	41-42	

Practical 1: Used of CREATE, ALTER, RENAME and DROP statement in the database tables (relations).

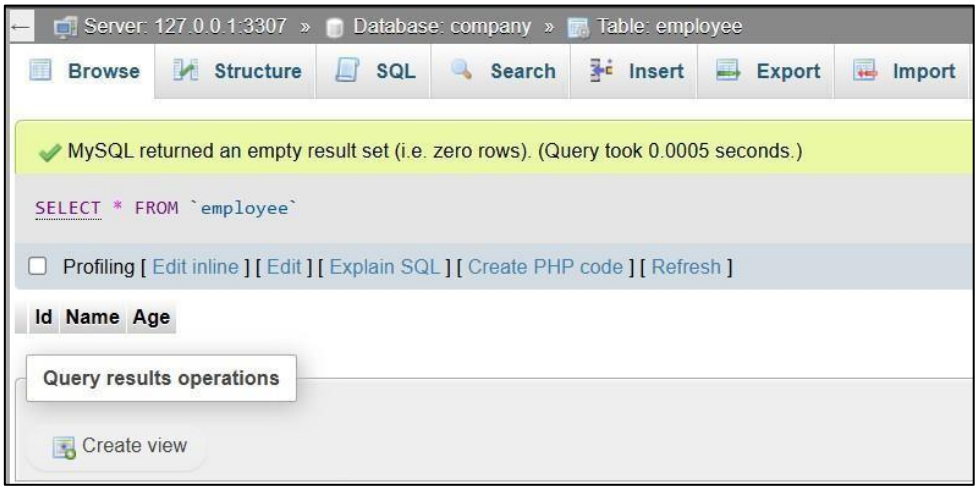
1. **CREATE TABLE Statement :**

   Definition: The CREATE TABLE statement is used to define a new table and its structure including columns and their data types.

   Syntax: CREATE TABLE table_name (column1 datatype, column2 datatype, ...);

   Example: CREATE TABLE employee(Id int PRIMARY KEY not null, Name varchar(50), Age int);

   OUTPUT:



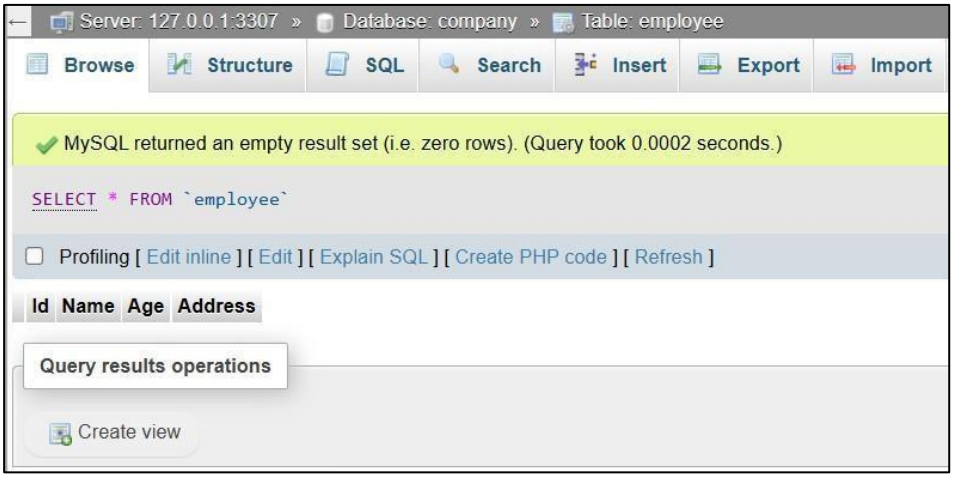
2. **ALTER TABLE Statement:**

   Definition: The ALTER TABLE statement is used to modify the structure of an existing table, such as adding, deleting, or modifying columns.

   Syntax: ALTER TABLE table_name ADD column_name datatype;

   Example: ALTER TABLE employee ADD Address varchar(50);

OUTPUT:



3. **RENAME TABLE Statement:**

Definition:

The RENAME TABLE statement is used to change the name of an existing table in a database. This operation helps in modifying table names without affecting the data inside the table.

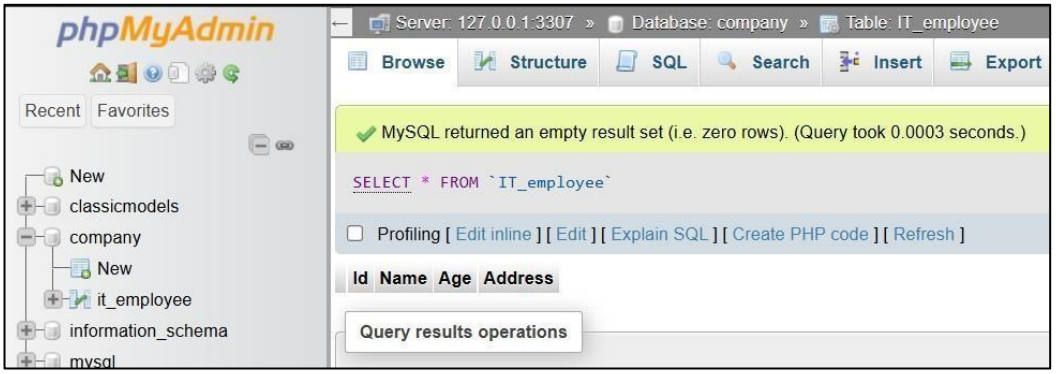
Syntax:

RENAME TABLE old_table_name TO new_table_name;

Example:

RENAME TABLE employee TO IT_employee;

OUTPUT:



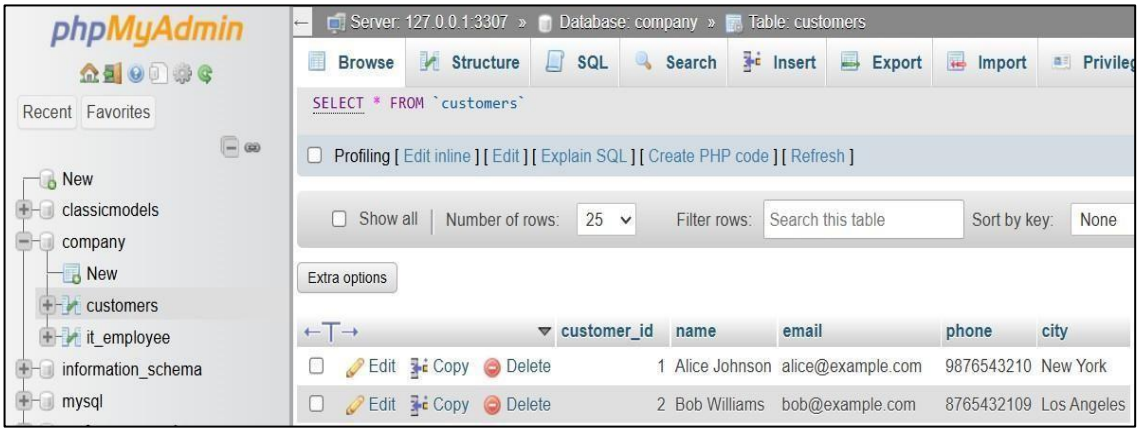
4. DROP TABLE Statement:

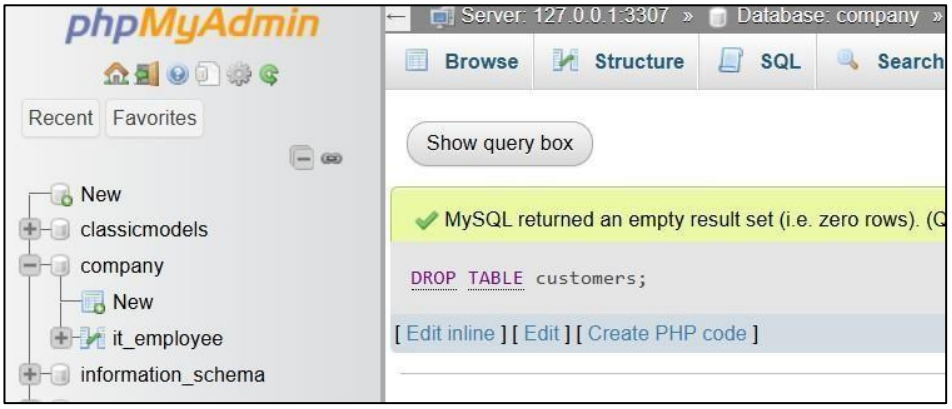
Definition: The DROP TABLE statement is used to delete an existing table and all of its data from the database.

Syntax: DROP TABLE table_name;

Example: DROP TABLE customers;

OUTPUT:





Practical 2: Used of INSERT INTO, DELETE and UPDATE statement in the database tables(relations).

1. **INSERT INTO Statement:**

   Definition: The INSERT INTO statement is used to add new records (rows) to a table. You can insert values into all columns or only specific ones.

   Syntax: INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);

   Example: INSERT INTO it_employee VALUES(111,"Sahil",21,"Rajpura");

   INSERT INTO it_employee VALUES(222,"Rahul",24,"Patiala");

   OUTPUT:

2. **DELETE Statement:**

   Definition: The DELETE statement removes one or more records from a table based on a specified condition. If no condition is provided, all rows are deleted.

   Syntax: DELETE FROM table_name WHERE condition;

   Example: DELETE FROM it_employee WHERE Age < 20;

OUTPUT:

3. **UPDATE Statement:**

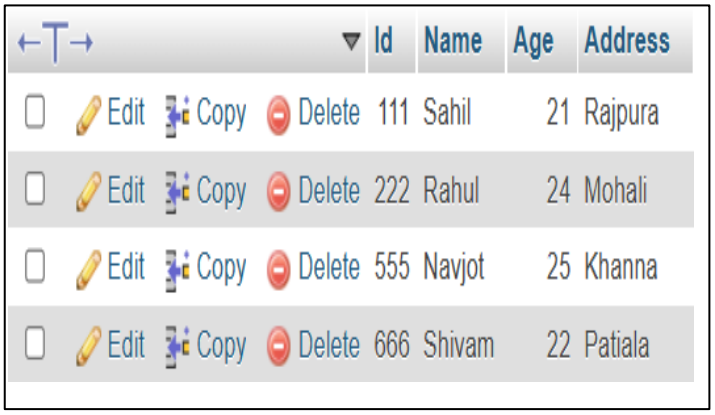
Definition: The UPDATE statement is used to modify existing records in a table. You can update one or more columns based on a condition.

Syntax: UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;

Example: UPDATE it_employee SET Address="Mohali" WHERE Id=222;

OUTPUT:


Server: 127.0.0.1:3307 » Database: company » Table: it_employee
Browse
Structure
SQL
Search
Insert
Export
Extra options
Id Name Age Address
Edit Copy Delete 111 Sahil 21 Rajpura
Edit Copy Delete 222 Rahul 24 Patiala
Edit Copy Delete 555 Navjot 25 Khanna
Edit Copy Delete 666 Shivam 22 Patiala


Id Name Age Address
Edit Copy Delete 111 Sahil 21 Rajpura
Edit Copy Delete 222 Rahul 24 Mohali
Edit Copy Delete 555 Navjot 25 Khanna
Edit Copy Delete 666 Shivam 22 Patiala

Practical 3: Use of simple select statement.

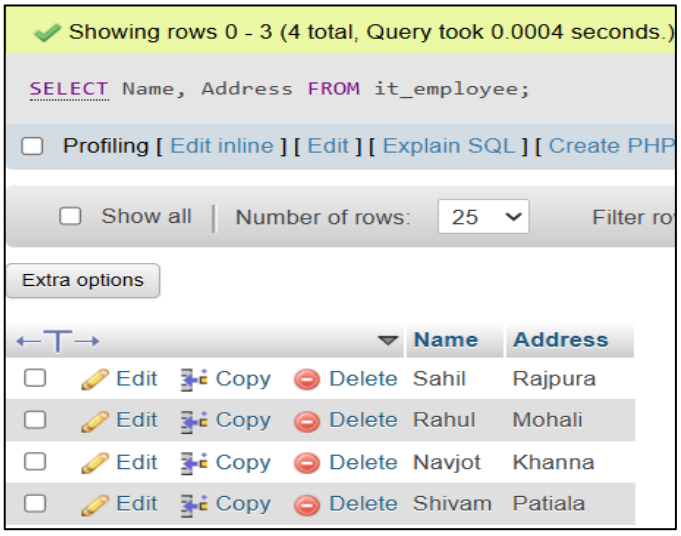
SELECT Statement:

Definition: The SELECT statement is used to retrieve data from one or more tables. It is the most commonly used SQL command.

Syntax: SELECT column1, column2 FROM table_name WHERE condition;

Example: SELECT Name, Address FROM it_employee;

OUTPUT:



Practical 4: Use of Select query on two relations.

Definition:

The SELECT statement is used to retrieve data from one or more tables (relations). When querying two tables, we typically use **JOIN operations** to combine related data based on common columns.

Syntax (Using INNER JOIN):

SELECT table1.column1, table2.column2, ...

FROM table1

INNER JOIN table2

ON table1.common_column = table2.common_column;

Example:

SELECT employees.name, employees.age, departments.dept_name, employees.salary

FROM employees

INNER JOIN departments ON employees.dept_id = departments.dept_id;

Department Table:

	dept_id	dept_name
Edit Copy Delete	1	IT
Edit Copy Delete	2	HR
Edit Copy Delete	3	Finance

Employees Table:

	emp_id	name	age	dept_id	salary
Edit Copy Delete	101	John Doe	30	1	60000.50
Edit Copy Delete	102	Jane Smith	28	2	50000.75
Edit Copy Delete	103	Mark Taylor	35	3	75000.00

OUTPUT:

name	age	dept_name	salary
John Doe	30	IT	60000.50
Jane Smith	28	HR	50000.75
Mark Taylor	35	Finance	75000.00

Practical 5: Use of nesting of queries.

Nested SELECT (Subquery):

Definition: A nested SELECT or subquery is a SELECT query placed inside another query to retrieve data based on the result of the inner query.

Syntax: SELECT column FROM table

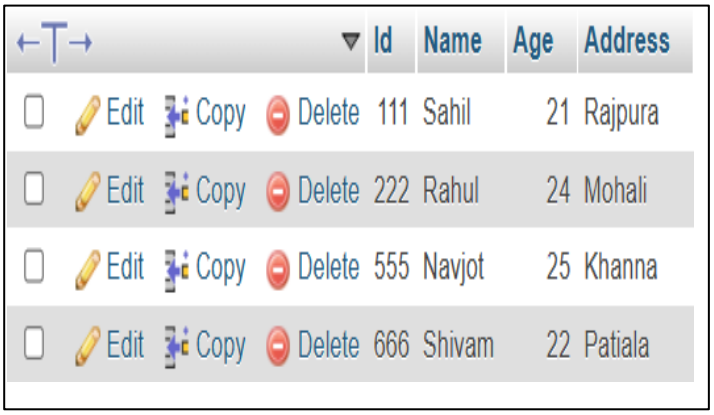
WHERE column IN (SELECT column FROM table WHERE condition);

Example:

SELECT name FROM it_employee

WHERE Id IN (SELECT Id FROM it_employee WHERE Id IN (111, 666));

OUTPUT:



	Id	Name	Age	Address
Edit Copy Delete	111	Sahil	21	Rajpura
Edit Copy Delete	222	Rahul	24	Mohali
Edit Copy Delete	555	Navjot	25	Khanna
Edit Copy Delete	666	Shivam	22	Patiala

	name
Edit Copy Delete	Sahil
Edit Copy Delete	Shivam

Practical 6: Use of Aggregate functions.

1. COUNT () Function:

   Definition: Returns the number of records in a table or column.

   Syntax: SELECT COUNT (column_name) FROM table_name;

   Example: SELECT COUNT(Id) FROM it_employee;

   OUTPUT:

COUNT(Id)
4

2. **SUM () Function:**

   Definition: Returns the total sum of a numeric column.

   Syntax: SELECT SUM (column_name) FROM table_name;

   Example: SELECT SUM(Salary) FROM it_employee;

   OUTPUT:

SUM(Salary)
260000

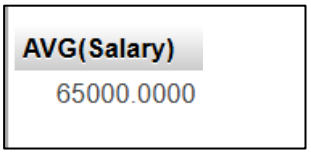
3. **AVG () Function:**

   Definition: Returns the average value of a numeric column.

   Syntax: SELECT AVG (column_name) FROM table_name;

Example: SELECT AVG(Salary) FROM it_employee;

OUTPUT:



AVG(Salary)
65000.0000

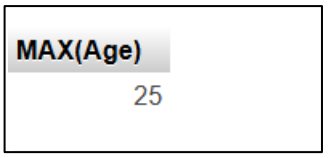
4. **MAX () Function:**

Definition: Returns the highest value in a column.

Syntax: SELECT MAX (column_name) FROM table_name;

Example: SELECT MAX(Age) FROM it_employee;

OUTPUT:



MAX(Age)
25

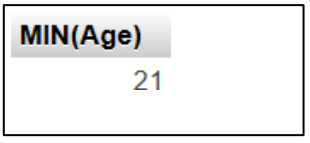
5. **MIN () Function:**

Definition: Returns the lowest value in a column.

Syntax: SELECT MIN (column_name) FROM table_name;

Example: SELECT MIN(Age) FROM it_employee;

OUTPUT:



MIN(Age)
21

Practical 7: Use of Substring comparison.

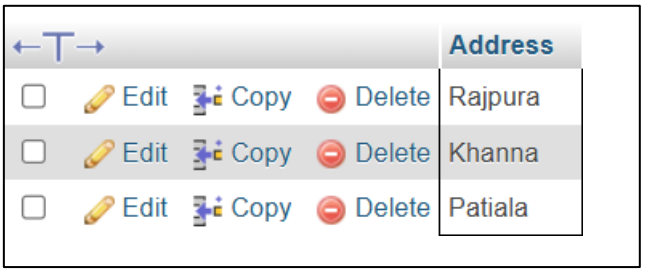
SUBSTRING comparison:

Definition: Used to compare part of strings.

Syntax: SELECT * FROM table_name WHERE SUBSTRING(column_name) LIKE 'Data%';

Example: SELECT Address FROM it_employee WHERE Address LIKE '%a';

OUTPUT:



Practical 8: Use of order by statement.

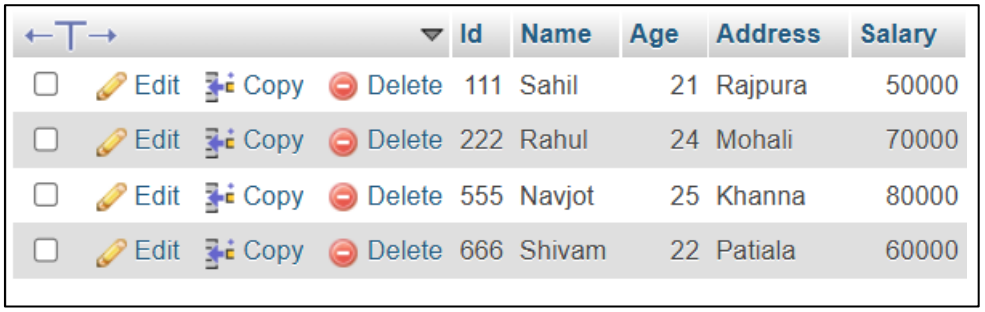
ORDER BY Statement:

Definition: The ORDER BY clause is used to sort the result set in ascending or descending order based on one or more columns.

Syntax: SELECT column1, column2, ... FROM table_name ORDER BY column_name [ASC|DESC];

Example: SELECT * FROM it_employee ORDER BY Age ASC;

OUTPUT:



	Id	Name	Age	Address	Salary
Edit Copy Delete	111	Sahil	21	Rajpura	50000
Edit Copy Delete	222	Rahul	24	Mohali	70000
Edit Copy Delete	555	Navjot	25	Khanna	80000
Edit Copy Delete	666	Shivam	22	Patiala	60000

	Id	Name	Age ▲ 1	Address	Salary
Edit Copy Delete	111	Sahil	21	Rajpura	50000
Edit Copy Delete	666	Shivam	22	Patiala	60000
Edit Copy Delete	222	Rahul	24	Mohali	70000
Edit Copy Delete	555	Navjot	25	Khanna	80000

Practical 9: Consider the following schema for a Library Database: BOOK (Book_id, Title, Publisher_Name, Pub_Year)

BOOK_AUTHORS (Book_id, Author_Name) PUBLISHER (Name, Address, Phone) BOOK_COPIES (Book_id, Branch_id, No-of_Copies) BOOK_LENDING (Book_id, Branch_id, Card_No, Date_Out, Due_Date) LIBRARY_BRANCH (Branch_id, Branch_Name, Address) Write SQL queries to

1. Retrieve details of all books in the library_id, title, name of publisher, authors, number of copies in each branch, etc.

2. Get the particulars of borrowers who have borrowed more than 3 books between Jan 2018 to Jun 2018

3. Delete a book in BOOK table. Update the contents of other tables to reflect this data manipulation operation.

4. Partition the BOOK table based on year of publication. Demonstrate its working with a simple query.

5. Create a view of all books and its number of copies that are currently available in the library.

1. **BOOK Table :**

```
CREATE TABLE BOOK (
   Book_id INT PRIMARY KEY,
   Title VARCHAR(255),
   Publisher_Name VARCHAR(100),
   Pub_Year YEAR
);


INSERT INTO BOOK (Book_id, Title, Publisher_Name, Pub_Year) VALUES
(1, 'Database Systems', 'Pearson', 2015),
(2, 'Operating Systems', 'McGraw-Hill', 2018),
(3, 'Machine Learning', 'Oxford', 2020);
```

OUTPUT:

Book_id	Title	Publisher_Name	Pub_Year
1	Database Systems	Pearson	2015
2	Operating Systems	McGraw-Hill	2018
3	Machine Learning	Oxford	2020

2. **BOOK_AUTHORS Table:**

```
CREATE TABLE BOOK_AUTHORS (
   Book_id INT,
   Author_Name VARCHAR(100),
   FOREIGN KEY (Book_id) REFERENCES BOOK(Book_id) ON DELETE
CASCADE
);

INSERT INTO BOOK_AUTHORS (Book_id, Author_Name) VALUES
(1, 'Raghu Ramakrishnan'),
(1, 'Johannes Gehrke'),
(2, 'Abraham Silberschatz'),
(2, 'Peter Baer Galvin'),
(3, 'Tom Mitchell');
```

OUTPUT:

Book_id	Author_Name
1	Raghu Ramakrishnan
1	Johannes Gehrke
2	Abraham Silberschatz
2	Peter Baer Galvin
3	Tom Mitchell

3. **PUBLISHER Table:**

```
CREATE TABLE PUBLISHER (
   Name VARCHAR(100) PRIMARY KEY,
   Address VARCHAR(255),
   Phone VARCHAR(15)
);

INSERT INTO PUBLISHER (Name, Address, Phone) VALUES
('Pearson', 'New York, USA', '1234567890'),
('McGraw-Hill', 'Boston, USA', '9876543210'),
('Oxford', 'London, UK', '5678901234');
```

OUTPUT:

Name	Address	Phone
McGraw-Hill	Boston, USA	9876543210
Oxford	London, UK	5678901234
Pearson	New York, USA	1234567890

4. **BOOK_COPIES Table:**

```
CREATE TABLE BOOK_COPIES (
Book_id INT,
Branch_id INT,
No_of_Copies INT,
FOREIGN KEY (Book_id) REFERENCES BOOK(Book_id)
);

INSERT INTO BOOK_COPIES (Book_id, Branch_id, No_of_Copies) VALUES
```

(1, 101, 5),
(1, 102, 3),
(2, 101, 2),
(3, 102, 4);

OUTPUT:

Book_id	Branch_id	No_of_Copies
1	101	5
1	102	3
2	101	2
3	102	4

5. **BOOK_LENDING Table:**

```
CREATE TABLE BOOK_LENDING (
   Book_id INT,
   Branch_id INT,
   Card_No INT,
   Date_Out DATE,
   Due_Date DATE,
   PRIMARY KEY (Book_id, Branch_id, Card_No, Date_Out),
   FOREIGN KEY (Book_id) REFERENCES BOOK(Book_id),
   FOREIGN KEY (Branch_id) REFERENCES LIBRARY_BRANCH(Branch_id)
);

INSERT INTO BOOK_LENDING (Book_id, Branch_id, Card_No, Date_Out,
Due_Date) VALUES
(1, 101, 1001, '2018-02-01', '2018-02-15'),
(2, 101, 1002, '2018-03-10', '2018-03-24'),
(3, 102, 1003, '2018-05-05', '2018-05-19'),
(1, 102, 1001, '2018-06-01', '2018-06-15'),
(2, 101, 1001, '2018-06-10', '2018-06-24'),
(3, 102, 1001, '2018-06-15', '2018-06-29');
```

OUTPUT:

Book_id	Branch_id	Card_No	Date_Out	Due_Date
1	101	1001	2018-02-01	2018-02-15
1	102	1001	2018-06-01	2018-06-15
2	101	1001	2018-06-10	2018-06-24
2	101	1002	2018-03-10	2018-03-24
3	102	1001	2018-06-15	2018-06-29
3	102	1003	2018-05-05	2018-05-19

6. **LIBRARY_BRANCH:**

```
CREATE TABLE LIBRARY_BRANCH (
   Branch_id INT PRIMARY KEY,
   Branch_Name VARCHAR(100),
   Address VARCHAR(255)
);

INSERT INTO LIBRARY_BRANCH (Branch_id, Branch_Name, Address)
VALUES
(101, 'Central Library', '123 Main St, New York'),
(102, 'West Side Library', '456 West St, Boston');
```

OUTPUT:

Branch_id	Branch_Name	Address
101	Central Library	123 Main St, New York
102	West Side Library	456 West St, Boston

Query 1: Retrieve full book details (with authors, publisher, copies).

```
SELECT B.Book_id, B.Title, P.Name AS Publisher_Name,
    GROUP_CONCAT(A.Author_Name SEPARATOR ', ') AS Authors,
    C.Branch_id, C.No_of_Copies
FROM BOOK B
JOIN PUBLISHER P ON B.Publisher_Name = P.Name
JOIN BOOK_AUTHORS A ON B.Book_id = A.Book_id
JOIN BOOK_COPIES C ON B.Book_id = C.Book_id
GROUP BY B.Book_id, C.Branch_id;
```

OUTPUT:

Book_id	Title	Publisher_Name	Authors	Branch_id	No_of_Copies
1	Database Systems	Pearson	Johannes Gehrke, Raghu Ramakrishnan	101	5
1	Database Systems	Pearson	Johannes Gehrke, Raghu Ramakrishnan	102	3
2	Operating Systems	McGraw-Hill	Abraham Silberschatz, Peter Baer Galvin	101	2
3	Machine Learning	Oxford	Tom Mitchell	102	4

Query 2: Borrowers who borrowed more than 3 books (Jan–Jun 2018).

```
SELECT L.Card_No, COUNT(L.Book_id) AS Books_Borrowed
FROM BOOK_LENDING L
WHERE L.Date_Out BETWEEN '2018-01-01' AND '2018-06-30'
GROUP BY L.Card_No
HAVING COUNT(L.Book_id) > 3;
```

OUTPUT:

Card_No	Books_Borrowed
1001	4

Query 3: Delete a book and update other tables.

```
DELETE FROM BOOK_AUTHORS WHERE Book_id = 1;
DELETE FROM BOOK_COPIES WHERE Book_id = 1;
DELETE FROM BOOK_LENDING WHERE Book_id = 1;
DELETE FROM BOOK WHERE Book_id = 1;
```

OUTPUT:

Book_id	Title	Publisher_Name	Pub_Year
1	Database Systems	Pearson	2015
2	Operating Systems	McGraw-Hill	2018
3	Machine Learning	Oxford	2020

Book_id	Title	Publisher_Name	Pub_Year
2	Operating Systems	McGraw-Hill	2018
3	Machine Learning	Oxford	2020

Query 4: Partition BOOK table by publication year.

```
ALTER TABLE book
PARTITION BY RANGE (Pub_Year) (
  PARTITION p_before_2000 VALUES LESS THAN (2000),
```

```
    PARTITION p_2000_2010 VALUES LESS THAN (2010),
    PARTITION p_2010_2020 VALUES LESS THAN (2020),
    PARTITION p_2020_onwards VALUES LESS THAN MAXVALUE
);

SELECT * FROM book PARTITION(p_2010_2020);
```

OUTPUT:

Book_id	Title	Publisher_Name	Pub_Year
2	Operating Systems	McGraw-Hill	2018
3	Machine Learning	Oxford	2020

Book_id	Title	Publisher_Name	Pub_Year
2	Operating Systems	McGraw-Hill	2018

Query 5: Create a view of all books with the number of available copies.

```
CREATE VIEW Available_Books AS
SELECT B.Book_id, B.Title, SUM(C.No_of_Copies) AS Total_Copies
FROM BOOK B
JOIN BOOK_COPIES C ON B.Book_id = C.Book_id
GROUP BY B.Book_id;
```

OUTPUT:

Book_id	Title	Total_Copies
2	Operating Systems	2
3	Machine Learning	4

Practical 10: Consider the following schema for Order Database: SALESMAN (Salesman_id, Name, City, Commission) CUSTOMER (Customer_id, Cust_Name, City, Grade, Salesman_id) ORDERS (Ord_No, Purchase_Amt, Ord_Date, Customer_id, Salesman_id) Write SQL queries to

1. Count the customers with grades above Amritsar's average.

2. Find the name and numbers of all salesmen who had more than one customer.

3. List all salesmen and indicate those who have and don't have customers in their cities (Use UNION operation.)

4. Create a view that finds the salesman who has the customer with the highest order of a day.

5. Demonstrate the DELETE operation by removing salesman with id 1000. All his orders must also be deleted.

Order Database Queries:

1. **SALESMAN Table:**

```
CREATE TABLE SALESMAN (
   Salesman_id INT PRIMARY KEY,
   Name VARCHAR(50) NOT NULL,
   City VARCHAR(50) NOT NULL,
   Commission DECIMAL(5,2) NOT NULL
);


INSERT INTO SALESMAN VALUES

(1000, 'John Doe', 'Amritsar', 0.15),

(1001, 'Jane Smith', 'Delhi', 0.12),
```

(1002, 'Robert Johnson', 'Mumbai', 0.10),

(1003, 'Alice Brown', 'Amritsar', 0.11),

(1004, 'David Wilson', 'Chennai', 0.14),

(1005, 'Emily Davis', 'Delhi', 0.13);

OUTPUT:

Salesman_id	Name	City	Commission
1000	John Doe	Amritsar	0.15
1001	Jane Smith	Delhi	0.12
1002	Robert Johnson	Mumbai	0.10
1003	Alice Brown	Amritsar	0.11
1004	David Wilson	Chennai	0.14
1005	Emily Davis	Delhi	0.13

2. **CUSTOMER Table:**

```
CREATE TABLE CUSTOMER (
   Customer_id INT PRIMARY KEY,
   Cust_Name VARCHAR(50) NOT NULL,
   City VARCHAR(50) NOT NULL,
   Grade INT,
   Salesman_id INT,
   FOREIGN KEY (Salesman_id) REFERENCES SALESMAN(Salesman_id)
);
```

INSERT INTO CUSTOMER VALUES

(2001, 'Customer A', 'Amritsar', 2, 1000),

(2002, 'Customer B', 'Delhi', 1, 1001),

(2003, 'Customer C', 'Mumbai', 3, 1002),

(2004, 'Customer D', 'Amritsar', 2, 1000),

(2005, 'Customer E', 'Chennai', 1, 1004),

(2006, 'Customer F', 'Delhi', 2, 1001),

(2007, 'Customer G', 'Mumbai', 3, 1002),

(2008, 'Customer H', 'Amritsar', 1, 1003);

OUTPUT:

Customer_id	Cust_Name	City	Grade	Salesman_id
2001	Customer A	Amritsar	2	1000
2002	Customer B	Delhi	1	1001
2003	Customer C	Mumbai	3	1002
2004	Customer D	Amritsar	2	1000
2005	Customer E	Chennai	1	1004
2006	Customer F	Delhi	2	1001
2007	Customer G	Mumbai	3	1002
2008	Customer H	Amritsar	1	1003

3. **ORDERS Table:**

```
CREATE TABLE ORDERS (
   Ord_No INT PRIMARY KEY,
   Purchase_Amt DECIMAL(10,2) NOT NULL,
   Ord_Date DATE NOT NULL,
   Customer_id INT,
   Salesman_id INT,
   FOREIGN KEY (Customer_id) REFERENCES CUSTOMER(Customer_id),
   FOREIGN KEY (Salesman_id) REFERENCES SALESMAN(Salesman_id)
);
```

```
INSERT INTO ORDERS VALUES

(3001, 1500.00, '2023-01-15', 2001, 1000),

(3002, 2500.00, '2023-01-15', 2002, 1001),
```

(3003, 1800.00, '2023-01-16', 2003, 1002),

(3004, 3500.00, '2023-01-16', 2004, 1000),

(3005, 1200.00, '2023-01-17', 2005, 1004),

(3006, 2800.00, '2023-01-17', 2006, 1001),

(3007, 4200.00, '2023-01-18', 2007, 1002),

(3008, 1900.00, '2023-01-18', 2008, 1003),

(3009, 3100.00, '2023-01-19', 2001, 1000),

(3010, 2200.00, '2023-01-19', 2003, 1002);

OUTPUT:

Ord_No	Purchase_Amt	Ord_Date	Customer_id	Salesman_id
3001	1500.00	2023-01-15	2001	1000
3002	2500.00	2023-01-15	2002	1001
3003	1800.00	2023-01-16	2003	1002
3004	3500.00	2023-01-16	2004	1000
3005	1200.00	2023-01-17	2005	1004
3006	2800.00	2023-01-17	2006	1001
3007	4200.00	2023-01-18	2007	1002
3008	1900.00	2023-01-18	2008	1003
3009	3100.00	2023-01-19	2001	1000
3010	2200.00	2023-01-19	2003	1002

Query 1: Count the customers with grades above Amritsar's average.

```
SELECT COUNT(*) AS Customer_Count
FROM CUSTOMER
WHERE Grade > (
```

```
    SELECT AVG(Grade)
    FROM CUSTOMER
    WHERE City = 'Amritsar'
);
```

OUTPUT:

Customer_Count
5

Query 2: Find the name and numbers of all salesmen who had more than one customer.

```
SELECT s.Salesman_id, s.Name, COUNT(c.Customer_id) AS Customer_Count
FROM SALESMAN s
JOIN CUSTOMER c ON s.Salesman_id = c.Salesman_id
GROUP BY s.Salesman_id, s.Name
HAVING COUNT(c.Customer_id) > 1;
```

OUTPUT:

Salesman_id	Name	Customer_Count
1000	John Doe	2
1001	Jane Smith	2
1002	Robert Johnson	2

Query 3: List all salesmen and indicate those who have and don't have customers in their cities (Use UNION operation)

```
SELECT s.Salesman_id, s.Name, 'Has customers in city' AS Status
FROM SALESMAN s
WHERE EXISTS (
   SELECT 1
   FROM CUSTOMER c
   WHERE c.Salesman_id = s.Salesman_id AND c.City = s.City
)
UNION
SELECT s.Salesman_id, s.Name, 'No customers in city' AS Status
FROM SALESMAN s
WHERE NOT EXISTS (
   SELECT 1
   FROM CUSTOMER c
   WHERE c.Salesman_id = s.Salesman_id AND c.City = s.City
);
```

OUTPUT:

Salesman_id	Name	Status
1000	John Doe	Has customers in city
1001	Jane Smith	Has customers in city
1002	Robert Johnson	Has customers in city
1003	Alice Brown	Has customers in city
1004	David Wilson	Has customers in city
1005	Emily Davis	No customers in city

Query 4: Create a view that finds the salesman who has the customer with the highest order of a day.

```
CREATE VIEW Highest_Order_Salesman AS
SELECT o.Ord_Date, s.Salesman_id, s.Name, MAX(o.Purchase_Amt) AS Highest_Order
FROM ORDERS o
JOIN SALESMAN s ON o.Salesman_id = s.Salesman_id
GROUP BY o.Ord_Date, s.Salesman_id, s.Name
HAVING MAX(o.Purchase_Amt) = (
   SELECT MAX(Purchase_Amt)
   FROM ORDERS
   WHERE Ord_Date = o.Ord_Date
);
```

OUTPUT:

Ord_Date	Salesman_id	Name	Highest_Order
2023-01-15	1001	Jane Smith	2500.00
2023-01-16	1002	Robert Johnson	1800.00
2023-01-17	1001	Jane Smith	2800.00
2023-01-18	1002	Robert Johnson	4200.00
2023-01-19	1002	Robert Johnson	2200.00

Query 5: Demonstrate the DELETE operation by removing salesman with id 1000. All his orders must also be deleted.

```
DELETE FROM ORDERS
WHERE Salesman_id = 1000;
```

```
DELETE FROM CUSTOMER
WHERE Salesman_id = 1000;
DELETE FROM SALESMAN
WHERE Salesman_id = 1000;
```

OUTPUT:

Customer_id	Cust_Name	City	Grade	Salesman_id
2002	Customer B	Delhi	1	1001
2003	Customer C	Mumbai	3	1002
2005	Customer E	Chennai	1	1004
2006	Customer F	Delhi	2	1001
2007	Customer G	Mumbai	3	1002
2008	Customer H	Amritsar	1	1003

Ord_No	Purchase_Amt	Ord_Date	Customer_id	Salesman_id
3002	2500.00	2023-01-15	2002	1001
3003	1800.00	2023-01-16	2003	1002
3005	1200.00	2023-01-17	2005	1004
3006	2800.00	2023-01-17	2006	1001
3007	4200.00	2023-01-18	2007	1002
3008	1900.00	2023-01-18	2008	1003
3010	2200.00	2023-01-19	2003	1002

Salesman_id	Name	City	Commission
1001	Jane Smith	Delhi	0.12
1002	Robert Johnson	Mumbai	0.10
1003	Alice Brown	Amritsar	0.11
1004	David Wilson	Chennai	0.14
1005	Emily Davis	Delhi	0.13

Practical 11: Write a PL/SQL code to add two numbers and display the result. Read the numbers during run time.

```
CREATE TABLE Add_Numbers ( num1 INT, num2 INT );
INSERT INTO Add_Numbers VALUES (10, 20);
INSERT INTO Add_Numbers VALUES (10, 20);
INSERT INTO Add_Numbers VALUES (14, 50);
INSERT INTO Add_Numbers VALUES (78, 37);
INSERT INTO Add_Numbers VALUES (76, 2);
INSERT INTO Add_Numbers VALUES (19, 75);
SELECT num1, num2, (num1 + num2) AS sum_result FROM
Add_Numbers;
```

OUTPUT:

num1	num2	sum_result
10	20	30
10	20	30
14	50	64
78	37	115
76	2	78
19	75	94

Practical 12: Write a PL/SQL code to find sum of first 10 natural numbers using while and for loop.

Using WHILE Loop:

```
DELIMITER //
CREATE PROCEDURE sum_first_10_natural()
BEGIN
   DECLARE i INT DEFAULT 1;
   DECLARE total INT DEFAULT 0;

   -- Using WHILE loop
   WHILE i <= 10 DO
      SET total = total + i;
      SET i = i + 1;
   END WHILE;
    SELECT CONCAT('Sum (WHILE loop): ', total) AS Result;
END //
DELIMITER ;
CALL sum_first_10_natural();
```

OUTPUT:

Result
Sum (WHILE loop): 55

Using FOR Loop:

```
DELIMITER //
CREATE PROCEDURE sum_with_repeat()
```

```
BEGIN
   DECLARE i INT DEFAULT 1;
   DECLARE total INT DEFAULT 0;
   REPEAT
      SET total = total + i;
      SET i = i + 1;
   UNTIL i > 10 END REPEAT;
   SELECT CONCAT('Sum (FOR loop): ', total) AS Result;
END //
DELIMITER ;
CALL sum_with_repeat();
```

OUTPUT:

Result
Sum (FOR loop): 55

Practical 13: Write a program to create a trigger which will convert the name of a student to upper case before inserting or updating the name column of student table.

```
CREATE TABLE student (
   student_id INT PRIMARY KEY,
   name VARCHAR(100),
   other_columns VARCHAR(100)
);
-- Create the trigger for INSERT
DELIMITER //
CREATE TRIGGER before_student_insert
BEFORE INSERT ON student
FOR EACH ROW
BEGIN
   SET NEW.name = UPPER(NEW.name);
END//
DELIMITER ;
-- Create the trigger for UPDATE
DELIMITER //
CREATE TRIGGER before_student_update
BEFORE UPDATE ON student
FOR EACH ROW
BEGIN
   SET NEW.name = UPPER(NEW.name);
END//
DELIMITER ;
```

```
INSERT INTO student (student_id, name) VALUES (1, 'jane smith');
INSERT INTO student (student_id, name) VALUES (2, 'john doe');
INSERT INTO student (student_id, name) VALUES (3, 'sam altman');
```

OUTPUT:

student_id	name	other_columns
1	JANE SMITH	*NULL*
2	JOHN DOE	*NULL*
3	SAM ALTMAN	*NULL*

Practical 14: Write a PL/SQL block to count the number of rows affected by an update statement using SQL%ROWCOUNT.

```
CREATE TABLE employees (
   employee_id INT PRIMARY KEY,
   first_name VARCHAR(50),
   last_name VARCHAR(50),
   salary DECIMAL(10,2),
   department_id INT
);

INSERT INTO employees VALUES
(1, 'John', 'Doe', 5000, 10),
(2, 'Jane', 'Smith', 6000, 20),
(3, 'Bob', 'Johnson', 5500, 10);

DELIMITER //
CREATE PROCEDURE update_employees_with_count()
BEGIN
   DECLARE rows_affected INT;
   UPDATE employees
   SET salary = salary * 1.1
   WHERE department_id = 10;
   SET rows_affected = ROW_COUNT();
   SELECT CONCAT('Rows affected: ', rows_affected) AS result
   COMMIT;
```

```
END //
DELIMITER ;
CALL update_employees_with_count();
```

OUTPUT:

result
Rows affected: 2

Practical 15: Write a PL/SQL block to increase the salary of all doctors by 1000.

```
CREATE TABLE doctors (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    specialization VARCHAR(100),
    salary DECIMAL(10,2) NOT NULL
);

INSERT INTO doctors (name, specialization, salary) VALUES
('Dr. Smith', 'Cardiology', 8000.00),
('Dr. Johnson', 'Pediatrics', 7500.00),
('Dr. Williams', 'Neurology', 9000.00);

DELIMITER //
CREATE PROCEDURE increase_doctor_salaries()
BEGIN
    DECLARE rows_affected INT;
    UPDATE doctors
    SET salary = salary + 1000;
    SET rows_affected = ROW_COUNT();
    SELECT CONCAT('Successfully increased salary for ', rows_affected, ' doctors')
AS result;
END //
DELIMITER ;
CALL increase_doctor_salaries();
```

OUTPUT:

id	name	specialization	salary
1	Dr. Smith	Cardiology	8000.00
2	Dr. Johnson	Pediatrics	7500.00
3	Dr. Williams	Neurology	9000.00

result
Successfully increased salary for 3 doctors

id	name	specialization	salary
1	Dr. Smith	Cardiology	9000.00
2	Dr. Johnson	Pediatrics	8500.00
3	Dr. Williams	Neurology	10000.00